



UNIVERSIDAD NACIONAL AUTÓNOMA DE  
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



**PREVIO N° 02**

**NOMBRE COMPLETO:** Del Toro Ortiz Juan Pablo Yasbeth

**N° de Cuenta:** 317031814

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 7

**SEMESTRE** 2026-2

**FECHA DE ENTREGA LÍMITE:** 13/02/2026

**CALIFICACIÓN:** \_\_\_\_\_

## Cuestionario Previo 2

¿Cómo funciona `glm::ortho`?

Creará una matriz de proyección ortográfica (paralela) lo que permite los objetos mantener su tamaño sin importar la distancia de la cámara. Se usa comúnmente en gráficos 2D o interfaces ya que no existe efecto de profundidad visual; recibe parámetros como `left`, `right`, `bottom`, `top`, `near` y `far` para definir límites de la forma de vista. (Learn OpenGL)

¿Cómo funciona `glm::perspective`?

Creará una matriz de proyección en perspectiva, que imita la forma en que el ojo humano percibe el mundo: los objetos lejanos se ven más pequeños. Recibe parámetros como el campo de visión (field of view o FOV), la relación del aspecto (aspect ratio), y los planos `near` y `far` (cerca y lejos). Esta matriz deforma la geometría para que, tras la división por perspectiva (que ocurre automáticamente en el pipeline), se genere el efecto de profundidad realista. (Learn OpenGL)

¿Cómo funciona `glViewport`?

Define el área de la ventana donde se dibujará la imagen, indicando su posición y tamaño en píxeles. Además recibe las coordenadas de la esquina inferior izquierda (`x`, `y`) y el ancho y alto del área. Básicamente le dice a OpenGL "todo lo calculado dibuja dentro de este rectángulo de la pantalla". Si cambia el tamaño de la ventana, es necesario ajustar los parámetros para evitar distorsiones. (StewieWhims `glViewport` Microsoft)



## ¿Cómo funciona `glm::translate`?

Genera una matriz de traslación que mueve un objeto en el espacio. Recibe dos parámetros: una matriz base (generalmente la matriz identidad) y un vector (`glm::vec3`) que indica cuánto desplazar los ejes  $X, Y, Z$ . Devuelve una nueva matriz que al multiplicarse por las coordenadas de los vértices en el shader, desplaza el objeto a la posición deseada. (CSSE351 Computer Graphics)

## ¿Cómo funciona `glm::rotate`?

Genera una matriz de rotación para girar un objeto alrededor de un eje. Recibe tres parámetros: una matriz base, un ángulo (generalmente en radianes) y un vector que define el eje de rotación (por ejemplo `glm::vec3(1,0,0)` para el eje  $X$ ). La matriz resultante al multiplicar los vértices, rota el objeto la cantidad especificada al rededor del eje indicado. (CSSE351 Computer Graphics)

## ¿Cómo funciona `glm::scale`?

Genera una matriz de escalado que modifica el tamaño de un objeto. Recibe una matriz base y un vector (`glm::vec3`) con los factores de escala para cada eje ( $X, Y, Z$ ). Si los 3 factores son iguales el escalado es uniforme; si son diferentes el objeto se estira o encoge de forma NO uniforme. Internamente construye una matriz con los factores en la diagonal principal, dejando de lado el último componente (el  $w$ ) en 1 para que la división por perspectiva no altere el efecto. (An introduction to GLMnet)



¿Qué son las variables Uniform dentro de GLSL y cómo se declaran y se mandan desde OpenGL a GLSL?

Son variables globales de solo lectura que permiten pasar datos constantes (como matrices de transformación, luces o tiempo) desde la CPU (OpenGL) a la GPU (shaders) y mantienen el mismo valor para todos los vértices o fragmentos durante una llamada de dibujo (draw call) (Zavala 2025).

Se declaran en el código del shader (Vertex o Fragment) usando la palabra clave uniform, generalmente fuera de las funciones y antes del main(); ejemplo:

```
#version 330 core
uniform mat4 U_modelViewMatrix; // matriz transformación
uniform vec3 U_lightPos; // posición de luz
void main() { ... }
```

Envío desde OpenGL (C++) a GLSL, este proceso requiere de 3 pasos fundamentales:

- 1) Enlazar el programa: Asegurarse de que el programa Shader está activo con `glUseProgram(ShaderProgram);`
- 2) Obtener la ubicación (Localitation). Buscar el ID de la variable uniform empleando usando la instrucción `glGetUniformLocation` por ejemplo:  
`GLint localitations = glGetUniformLocation(ShaderProgram, "U_lightPos");`
- 3) Actualizar el valor: Enviar el dato a esa ubicación  
`glUniform3f(1, 2, 3, 4, 5, 6, 7, 8, 9);` ó  
`glUniformMatrix4fv(2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32);`

Ejemplo:

```
glUniform3f(location, 1.0f, 5.0f, 2.0f);
```



## ¿Cómo funciona dentro de GLSL `gl_Position`?

Es una variable integrada (build-in) de tipo `vec4` (vector de 4 componentes:  $x, y, z, w$ ) en el vertex shader. Es la salida principal del vertex shader, responsable de definir la posición del vértice en el espacio de recorte.

Luego OpenGL automáticamente toma este valor y realiza la división por perspectiva (convirtiendo a coordenadas normalizadas de dispositivo, NDC) y lo usa básicamente para determinar donde aparece el vértice en la pantalla tras aplicar `glViewport` siendo el puente entre lo calculado por el shader y lo visualizado en pantalla. (Skorpius n.d)

## ¿Cómo funciona la variable `extern`?

La palabra clave `extern` se usa para indicar que una variable está definida en otro archivo, permite compartir variables globales entre distintos archivos fuente sin redefinirlas.

En versiones antiguas de GLSL se usaba la palabra clave `extern` para declarar variables que podían ser modificadas desde fuera del shader, pero actualmente está en desuso ya que su función fue reemplazada por la palabra clave `uniform`, que es el estándar actual para comunicar datos desde la aplicación al shader. (Learn OpenGL - Advanced GLSL)

# Conclusión:

Este cuestionario me ayudó a comprender mejor cómo se aplican las transformaciones y proyecciones dentro de la computación gráfica. Al analizar funciones como `glm::ortho`, `glm::perspective` y `glViewport`, puede entender cómo se define la forma en que los objetos se visualizan en la pantalla. Asimismo, el uso de transformaciones como traslación, rotación y escalado me permitió identificar cómo se manipulan los objetos de dentro de una escena.

Además luego de revisar conceptos como las variables uniform, `gl_Position` y `extern`, logré tener una idea más clara de cómo se comunican los datos entre OpenGL y GLSL y el programa principal, lo que me da una visión más completa del funcionamiento general del proceso de renderizado.

## Bibliografía

- LearnOpenGL - Advanced GLSL. (n.d.). <https://learnopengl.com/Advanced-OpenGL/Advanced-GLSL>
- Skorpis. (n.d.). In WebGL what are the differences between an attribute, a uniform, and a varying variable? Stack Overflow. <https://stackoverflow.com/questions/17537879/in-webgl-what-are-the-differences-between-an-attribute-a-uniform-and-a-varying>
- Zavala, F. (2025, October 4). Buffers y Uniforms en OpenGL. Código En Llamas | Programación, Algoritmos Y Software Explicado Paso a Paso. <https://codigoenllamas.com/buffers-y-uniforms-en-opengl>
- LearnOpenGL - Coordinate Systems. (n.d.). <https://learnopengl.com/Getting-started/Coordinate-Systems>
- CSSE351 Computer Graphics. (n.d.). <https://www.rose-hulman.edu/class/csse/csse351-old/m3/Using%20transforms/>
- Stevewhims. (n.d.). Función `glViewport` (GL.h) - Win32 apps. Microsoft Learn. <https://learn.microsoft.com/es-mx/windows/win32/opengl/glviewport>
- An Introduction to `glmnet`. (n.d.). <https://glmnet.stanford.edu/articles/glmnet.html>
- GeeksforGeeks. (s. f.). Understanding "extern" keyword in C. <https://www.geeksforgeeks.org/understanding-extern-keyword-in-c/>